

Word Embeddings: Dense Word Representations

Natural Language Processing

Daniel Ortiz Martínez

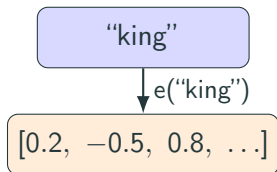
Table of Contents

1. Introduction
2. Distributional Semantics
3. Word Embeddings
4. Word2vec
5. Glove
6. Evaluating Word Embeddings: Analogy Tasks
7. The Gensim Library
8. The Gigaword Corpus

Introduction

Why Is Text Hard for Computers?

- Computers natively operate on **numbers**, not symbols
- Words carry **meaning**, **context**, and **relationships**
- The same concept can be expressed in many ways:
“car”, “automobile”, “vehicle”
- We need a **numeric representation** that preserves semantics



One-Hot Representation

- We can learn a vocabulary mapping that maps strings to integers and we can use it to construct “one hot encoded” words
- We denote by o the function creating the one hot encoding of a word:

$$o(\text{cat}) = (0, \dots, 0, \overset{\text{cat}}{1}, \overset{\text{dog}}{0}, \dots, 0)$$

$$o(\text{dog}) = (0, \dots, 0, 0, \overset{\text{cat}}{0}, \overset{\text{dog}}{1}, \dots, 0)$$

- In general let us denote by o_j^V the one hot vector induced by a vocabulary V that activates position j :

$$o_j^V = (0, \dots, 0, \overset{j}{1}, 0, \dots, 0) \in \mathbb{R}^V$$

One-Hot Representation Problems

- The distance between 2 different words (one hot encoded vectors) is always 1
- Does it make sense that $d(\text{cat}, \text{dog}) = d(\text{cat}, \text{table})$? Not really
- We want to learn a function e (embedding) that maps words to continuous real value numbers such that:

$$d(e(\text{cat}) - e(\text{dog})) < d(e(\text{cat}) - e(\text{table}))$$

$o(\text{cat}) = (0, \dots, 0, \overset{\text{cat}}{1}, \overset{\text{dog}}{0}, \overset{\text{table}}{0}, \overset{\text{chair}}{0}, \dots, 0)$

$e(\text{cat}) = (0.76, 0.54, \dots, 0.10, \dots, 0.10, 10)$

$o(\text{dog}) = (0, \dots, 0, \overset{\text{cat}}{0}, \overset{\text{dog}}{1}, \overset{\text{table}}{0}, \overset{\text{chair}}{0}, \dots, 0)$

$e(\text{dog}) = (0.70, 0.40, \dots, 0.10, \dots, 0.10, 0.10)$

$o(\text{table}) = (0, \dots, 0, \overset{\text{cat}}{0}, \overset{\text{dog}}{0}, \overset{\text{table}}{1}, \overset{\text{chair}}{0}, \dots, 0)$

$e(\text{table}) = (0.10, 0.10, \dots, 0.10, \dots, 0.50, 0.60)$

$o(\text{chair}) = (0, \dots, 0, \overset{\text{cat}}{0}, \overset{\text{dog}}{0}, \overset{\text{table}}{0}, \overset{\text{chair}}{1}, \dots, 0)$

$e(\text{table}) = (0.10, 0.10, \dots, 0.10, \dots, 0.60, 0.80)$

Distributional Semantics

The Distributional Hypothesis

Firth 1957

"You shall know a word by the company it keeps."

- Words that appear in **similar contexts** tend to have **similar meanings**
- *"The ? sat on the mat" → cat, dog, child, ...*
- *"She bought a new ?" → car, phone, dress, ...*

Key idea: use context to build vector representations

Co-occurrence Matrices

Corpus: *"I like NLP. I like deep learning. NLP is fun."*

Window size = 1:

	I	like	NLP	deep	learning	fun
I	0	2	0	0	0	0
like	2	0	1	1	0	0
NLP	0	1	0	0	0	1
deep	0	1	0	0	1	0

Limitations:

- Matrix size: $|V| \times |V|$ — very large and sparse
- Frequent words dominate (e.g., *"the"*, *"a"*)
- Dimensionality reduction (SVD/LSA) helps, but is expensive

Word Embeddings

What Is a Word Embedding?

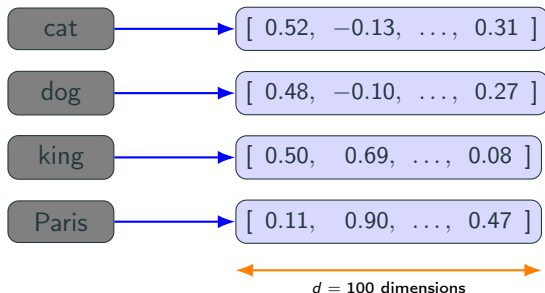
A **word embedding** is a function that maps each word in a vocabulary to a dense, real-valued vector in a low-dimensional space:

$$\phi : V \longrightarrow \mathbb{R}^d \quad \text{with } d \ll |V|$$

The mapping is learned from text so that **geometric proximity reflects semantic similarity**: words with related meanings end up close in vector space

Word Embeddings: Properties

- **Dense:** every dimension carries information
- **Distributed:** meaning spread across all dimensions
- **Learned:** vectors emerge from the statistics of large corpora
- **Transferable:** pre-trained vectors can be reused across many tasks



Training Word Embeddings: Predicting Nearby Words

- To get a dense representation for words, techniques such as Word2vec (Mikolov et al. 2013b) or Glove (Pennington et al. 2014) learn a mapping that *embeds* words to dense vectors of a pre-fixed dimension, which we call *embedding dimension*
- Given a corpus, word embedding techniques are based on solving a specific learning task
 - Word2vec uses a neural network to predict nearby words of a *center* or *pivot* word
 - Glove uses matrix factorization to predict global word co-occurrences

Word2vec

Basic Word Embeddings Methods

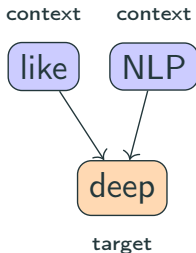
The Word2vec paper proposes two learning tasks to learn word embeddings:

- Continuous bag-of-words (**CBOW**): input is a bunch of words in a sentence, output is one word of the sentence that is *masked*
- Continuous skip-gram (**Skip-gram**): input is a word in a sentence, the output is a bunch of words that are next to the input word (also known as the context words)

Word2Vec: CBOW and Skip-gram

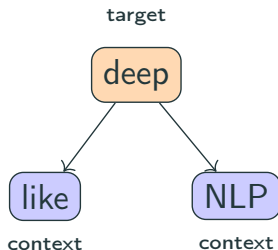
CBOW

Predicts target from context



Skip-gram

Predicts context from target

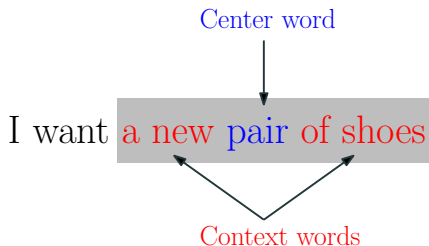


- Word2vec should not be seen as a single algorithm but more as software package to learn word embeddings
- The package has two distinct models:
 - CBOW
 - Skip-Gram
- Word2Vec implements ideas for fast learning with big vocabularies:
 - **Negative Sampling:** Allows fast normalization of the softmax
 - **Hierarchical Softmax:** Allows faster evaluation with $O(\log n)$ time instead of $O(n)$

The package also implements relevant preprocessing of the text:

- **Dynamic context windows:** words that are near to the target (or center) word are more important than other words that are far away from the target (or center) word
- **Subsampling:** Used to counter the imbalance between rare and frequent words

Definitions for CBOW



Context size = $C = 2$

Window size = $2 \cdot C + 1 = 5$

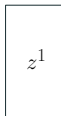
Transforming a Corpus to a Tabular Dataset

- In order to learn the embeddings, a sliding window is passed over each sentence, generating a set of training examples
- Example: Consider the sentence “I want a new pair of shoes”, $C=2$

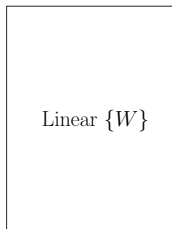
Sliding Window	Window	Input	Output
I want a new pair of shoes	[I, want, a, new, pair]	[I, want, new, pair]	a
I want a new pair of shoes	[want, a, new, pair, of]	[want, a, pair, of]	new
I want a new pair of shoes	[a, new, pair, of, shoes]	[a, new, of, shoes]	pair

CBOW: Notation

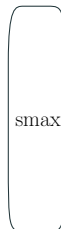
Preactivation Values



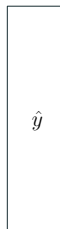
Linear Layer with
Learnable Parameters



Softmax Layer



Output Predictions

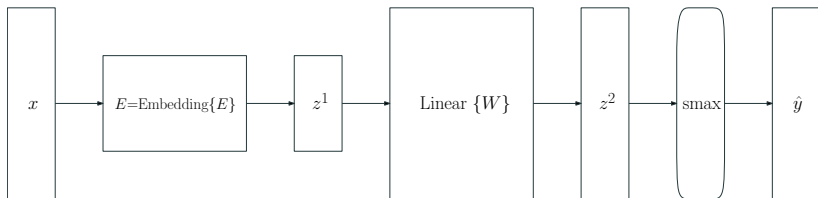


$$\text{smax} = \left(\frac{e^{z_1}}{\sum_{j=1}^V e^{z_j}}, \dots, \frac{e^{z_v}}{\sum_{j=1}^V e^{z_j}} \right)$$

CBOW: Neural Network Architecture

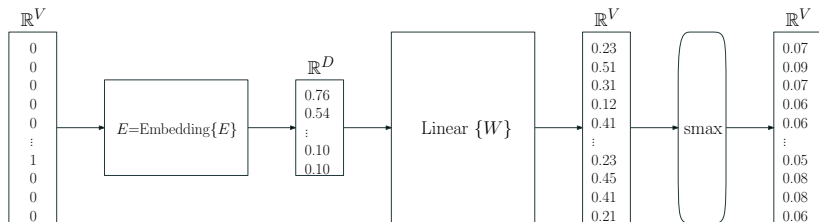
- Consider the following input to the CBOW model:

$$x = o(w_{t-2}) + o(w_{t-1}) + o(w_{t+1}) + o(w_{t+2}) = [0, 0, \dots, \overset{w_{t-2}}{1}, \dots, \overset{w_{t-1}}{1}, \dots, \overset{w_{t+1}}{1}, \dots, \overset{w_{t+2}}{1}, 0, 0]$$



CBOW Word Embeddings Placement in the Model

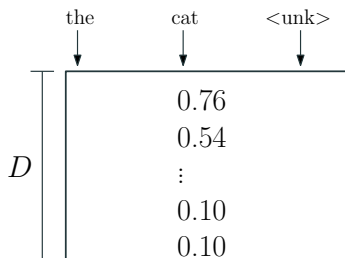
- The weight matrix, E , has one column per each vocabulary word
- Column j of the matrix, $E[:,j] := e_j$, contains a dense vector of shape D , which can be used as a word embedding for word in position j
- D is the dimensionality of the word embedding and a hyperparameter of the algorithm



CBOW Word Embedding

- If we multiply a one hot vector times E , that is, $E \cdot o_j$, we get $E[:, j]$
- Vocabulary $A^* \rightarrow \mathbb{N}$ maps strings (from Kleene closure of an alphabet A) to V positions

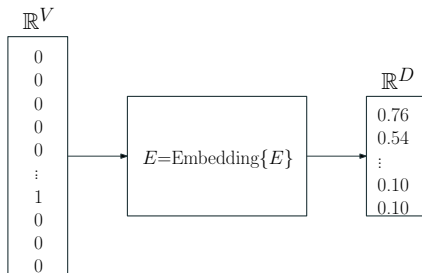
$$e(\text{cat}) := E \cdot o_{\text{vocab}(\text{cat})}^V = E[:, \text{vocab}(\text{cat})]$$



CBOW Forward Pass: First Layer

- The first embedding Layer $E : \mathbb{R}^V \rightarrow \mathbb{R}^D$ maps vectors of size V to D dimensional vectors
 - **Input:** $x = o(w_{t-2}) + o(w_{t-1}) + o(w_{t+1}) + o(w_{t+2})$
 - **Output:** $E(x) = \frac{1}{2C} (E \cdot o(w_{t-2}) + E \cdot o(w_{t-1}) + E \cdot o(w_{t+1}) + E \cdot o(w_{t+2}))$
- Note that the previous expression can be written as^a

$$E(x) = \frac{1}{2C} (E[:, \text{vocab}(w_{t-2})] + E[:, \text{vocab}(w_{t-1})] + E[:, \text{vocab}(w_{t+1})] + E[:, \text{vocab}(w_{t+2})])$$



^aMatrix operations are no longer used, the relevant columns of E are retrieved instead.

Embedding Layer versus Linear Layer

```
import timeit
import torch

# Initialize variables
num_embeddings = 10000
embedding_dim = 200
E = torch.nn.Embedding(num_embeddings, embedding_dim)

# Prepare input for the embedding layer
vocab = {'a':0, 'house':1, 'i':2}
x = torch.tensor([vocab['house'], vocab['i']])

# Prepare input for the dense layer
x_onehot = torch.zeros(num_embeddings)
x_onehot[vocab['house']] = 1
x_onehot[vocab['i']] = 1

# Create linear layer equivalent to embedding layer
E_trans_weight = E.weight.transpose(0, 1)
E_linear = torch.nn.Linear(num_embeddings, embedding_dim, bias=False)
E_linear.weight = torch.nn.Parameter(E_trans_weight)

# Calculate time
num_loop = 100000
time_embed = timeit.timeit("E.forward(x).sum(axis=0)", number=num_loop, globals=globals())
print("Embedding layer time:", time_embed/num_loop)
time_linear = timeit.timeit("E_linear.forward(x_onehot)", number=num_loop, globals=globals())
print("Linear layer time:", time_linear/num_loop)
print("Ratio:", time_linear/time_embed)

# Check the outputs are the same
torch.testing.assert_close(E_linear.forward(x_onehot), E.forward(x).sum(axis=0))
```

Embedding Layer versus Linear Layer: Output

```
Embedding layer time: 7.889608349942136e-06  
Linear layer time: 7.09692469699803e-05  
Ratio: 8.995281365329218
```

- Embedding layer is almost one order of magnitude faster
- The embedding layer works as a lookup table while the linear layer needs to compute matrix operations

CBOW Forward Pass: First Layer Example

- We have defined $E : \mathbb{R}^V \rightarrow \mathbb{R}^D$ to be

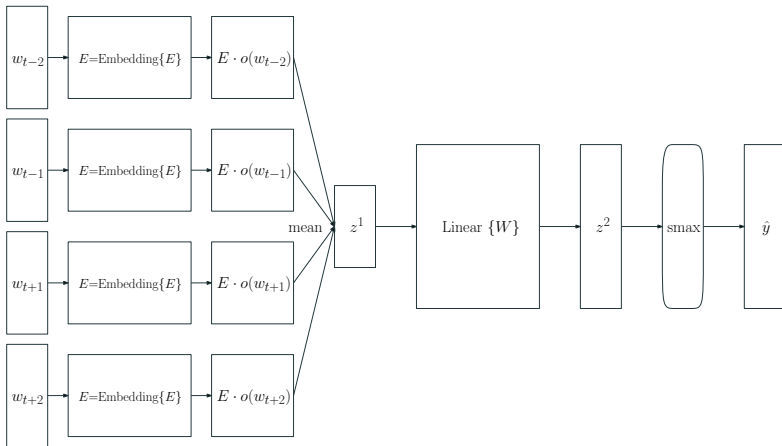
$$E(x) = \frac{1}{2C} (E \cdot o(w_{t-2}) + E \cdot o(w_{t-1}) + E \cdot o(w_{t-1} + E \cdot o(w_{t+2})))$$

- Consider the following example:
 - Sentence and sliding window: “ I love books because I love learning”
 - vocab = {am:0, because:1, happy:2, I:3, love:4, learning:5}
- The input for E in this example would be:

	I	love	because	I
am	0	0	0	0
because	0	0	1	0
happy	0	0	0	0
I	1	0	0	1
love	0	1	0	0
learning	0	0	0	0

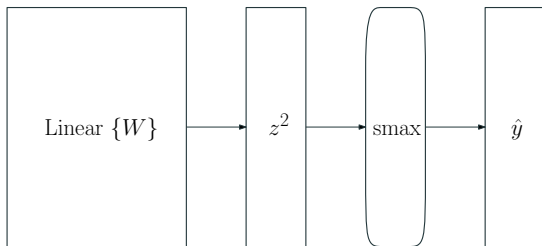
CBOW Forward Pass: Alternative View of First Layer

- This diagram emphasizes that the model is not exactly a standard MLP. In the forward pass examples are “forwarded” with the same weight matrix E and the results are aggregated with a mean



CBOW Forward Pass: Second Layer

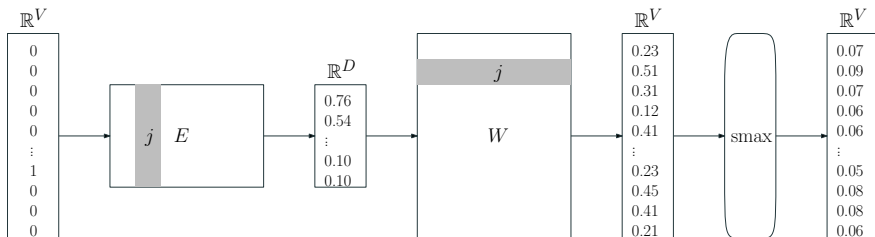
- The second layer takes the mean vector over the activated columns of E considered in the training example and passes the signal over a linear layer $W : \mathbb{R}^D \rightarrow \mathbb{R}^V$
- Then the output of the linear layer, z_2 , passes over a softmax



- Initialization step:
 1. Iterate over all the corpus to find the words in the vocabulary
 2. Build a vocab mapping that assigns a different integer to every word
- For a given sentence, select all possible windows. For each window:
 1. Compute the embedding layer activation of the input $z^1 = E(x)$
 2. Generate output scores $z^2 = W \cdot z^1$
 3. Turn scores into probabilities by calculating $\hat{y} = \text{smax}(z)$
 4. Compute the gradient of the cross-entropy loss
 5. Update the weights using gradient descent

CBOW Word Embedding Extraction

- After learning we have two matrices E and W
- E contains word embeddings as columns
- W contains word embeddings as rows
- We can extract the word embeddings as a mean over the two matrices
- Alternatively, we can just keep the column in the embedding matrix



Improving Training Efficiency with Negative Sampling

- The second layer takes the mean vector over the activated columns of E considered in the training example and passes the signal over a Linear Layer
- Then the output of the linear layer z^2 passes over a Softmax.
- Softmax step is very costly due to the computation of $\sum_{j=1}^V e^{z_j}$
- To alleviate this problem the training objective function is modified

Improving Training Efficiency with Negative Sampling

- Basic assumption: for each training sample, only a small percentage of weights will need to be updated
- Updates will be focused on the words of the context window and some random words outside the window, which constitute *negative samples*
 - Maximize probability to predict center word from context words
 - Minimize probability to predict center word from negative samples
- Negative samples are generated by a noise distribution

Word2Vec: Linear Analogies

The geometry of the embedding space encodes **semantic relationships**:

$$\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$$

$$\vec{\text{Paris}} - \vec{\text{France}} + \vec{\text{Italy}} \approx \vec{\text{Rome}}$$

Types of captured relations:

- Gender: *man* / *woman*
- Capitals: *country* / *city*
- Tense: *walk* / *walked*
- Superlatives: *big* / *biggest*



Glove

GloVe: Global Vectors for Word Representation

- Word2Vec uses local context windows
- GloVe uses global co-occurrence statistics: aggregated over the entire corpus
- GloVe learns to predict such co-occurrence statistics:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\vec{w}_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where:

- $f(X_{ij})$: weighting function to down-weight very frequent pairs
- $\vec{w}_i$: word vector
- b_i : scalar bias for word vector
- $\tilde{w}_j$: context vector
- $\tilde{b}_j$: scalar bias for context vector
- X_{ij} : number of times word j appears in context of word i

GloVe Training – Step 1: Building the Co-occurrence Matrix

Input: the entire corpus (e.g. Gigaword + Wikipedia, ~6B tokens)

For every word w_i and every word w_j appearing within a context window:

$$X_{ij} += \frac{1}{d}$$

where d is the distance between w_i and w_j (closer words contribute more)

Result: a symmetric matrix $X \in \mathbb{R}^{V \times V}$

- $V \approx 400,000$ for glove-wiki-gigaword-100
- Very **sparse** (most pairs never co-occur)
- Computed **once**, then used for training

GloVe Training – Step 2: The Objective Function

Goal: find word vectors $\vec{w}_i \in \mathbb{R}^d$ and context vectors $\tilde{w}_j \in \mathbb{R}^d$ such that their dot product approximates the log co-occurrence count:

$$\vec{w}_i^T \tilde{w}_j \approx \log X_{ij}$$

This is turned into a weighted least squares problem:

$$J = \sum_{\substack{i,j=1 \\ X_{ij}>0}}^V \underbrace{f(X_{ij})}_{\text{weight}} \left(\underbrace{\vec{w}_i^T \tilde{w}_j + b_i + \tilde{b}_j}_{\text{predicted log count}} - \underbrace{\log X_{ij}}_{\text{observed}} \right)^2$$

GloVe Training – Step 2: The Objective Function (Cont.)

Weighting function $f(X_{ij})$:

$$f(x) = \begin{cases} \left(\frac{x}{x_{\max}}\right)^{\alpha} & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

Typical values: $x_{\max} = 100$, $\alpha = 3/4$.

Why weight?

- Very frequent pairs (e.g. *the-of*) carry little meaning
- Very rare pairs are noisy
- f down-weights both extremes

Optimizer: AdaGrad (adaptive gradient descent)

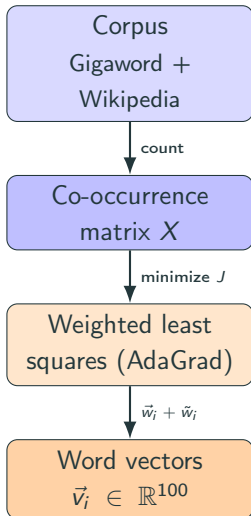
- Iterates over all non-zero (i, j) pairs in X
- Updates $\vec{w}_i, \tilde{w}_j, b_i, \tilde{b}_j$ to reduce J
- No backpropagation, no hidden layers, no activations
- Parallelizable (pairs are **independent**)

Final word vector:

$$\vec{v}_i = \vec{w}_i + \tilde{w}_i$$

Adding the two roles (word and context) empirically gives better results than using either alone

GloVe Training: Overview



GloVe vs. Word2Vec

	Word2Vec	GloVe
Training signal	Local context windows	Global co-occurrence matrix
Objective	Predictive (neural)	Matrix factorization
Scalability	Very fast (SGD)	Fast (weighted least squares)
Analogy tasks	Strong	Strong (often slightly better)
Interpretability	Lower	Slightly higher
OOV words	Not handled	Not handled

In practice: both produce high-quality embeddings; choice often depends on the downstream task and available resources

Evaluating Word Embeddings: Analogy Tasks

Evaluating Embeddings: Analogy Tasks

Idea: “*A is to B as C is to ?*” solved entirely by vector arithmetic

$$\vec{D} \approx \vec{B} - \vec{A} + \vec{C} \quad \Rightarrow \quad D = \arg \max_{w \notin \{A, B, C\}} \cos(\vec{w}, \vec{B} - \vec{A} + \vec{C})$$

A	B	C	Expected D
man	king	woman	queen
France	Paris	Italy	Rome

The Google Analogy Benchmark & Limitations

Google Analogy Dataset ([Mikolov et al. 2013a](#)):

- ~19,500 analogy questions
- Semantic subset: capitals, currencies, genders, family relations (~8,900 questions)
- Syntactic subset: verb tenses, plurals, comparatives (~10,600 questions)
- Metric: accuracy (the top-1 retrieved word must exactly match the expected answer)

Known limitations

- Only tests **linear** relationships
- Sensitive to **vocabulary coverage**: OOV words are skipped
- **Single gold answer**: ignores valid synonyms
- High analogy score $\neq$ good downstream task performance

The Gensim Library

Gensim is an open-source Python library for **unsupervised topic modelling and natural language processing**, with a focus on large-scale text processing and word vector models

Key features:

- Train **Word2Vec**, **FastText**, and **Doc2Vec** from scratch
- Load and query **pre-trained embeddings** (GloVe, Word2Vec, ...)
- Memory-efficient: streams data from disk, no need to load the full corpus into RAM
- Simple, consistent API across all models

Loading Word2Vec with Gensim

```
import gensim.downloader as api

# Downloads (around 1.6 GB) and caches the vectors locally
model = api.load("word2vec-google-news-300")

print(len(model))          # 3000000    (vocabulary size)
print(model.vector_size)    # 300        (embedding dimensions)
```

Loading GloVe with Gensim

```
import gensim.downloader as api

# Downloads (around 170 MB) and caches the vectors locally
model = api.load("glove-wiki-gigaword-100")

print(len(model))          # 400000  (vocabulary size)
print(model.vector_size)   # 100     (embedding dimensions)
```

The Gigaword Corpus

Introducing the Corpus: English Gigaword

What is Gigaword?

- A large archive of **newswire text** in English
- Sources: AP, NYT, AFP, Xinhua, ...
- $\approx$ **4.3 billion tokens** across millions of articles
- Time span: 1994–2010
- Domains: politics, economics, sports, world news

Why it matters:

- Rich vocabulary, formal register
- Biases of the source (news media)
- Standard NLP benchmark corpus

- **Vocabulary:** 400,000 tokens
- **Dimensions:** 100 (also available: 50, 200, 300)
 - Good balance: expressiveness vs. memory
 - Suitable for downstream tasks without GPU
- **Each word** encoded as a dense vector in $\mathbb{R}^{100}$
- Vectors are pre-trained (no training needed)
- Distributed by Stanford NLP Group (free to use)

Word	Vector (first 4 dims)
king	[0.50, 0.69, -0.08, ...]
queen	[0.37, 0.61, -0.05, ...]
man	[0.42, 0.30, 0.01, ...]
woman	[0.29, 0.38, -0.03, ...]
Paris	[0.11, 0.90, 0.47, ...]



Firth, J. (1957). *A synopsis of linguistic theory 1930-1955*. Studies in Linguistic Analysis, Philological. Longman.



Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean (2013a). *Efficient Estimation of Word Representations in Vector Space*. arXiv: 1301.3781 [cs.CL]. URL: <https://arxiv.org/abs/1301.3781>.



Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean (2013b). “Efficient Estimation of Word Representations in Vector Space”. In: *CoRR* abs/1301.3781. URL: <http://dblp.uni-trier.de/db/journals/corr/corr1301.html#abs-1301-3781>.



Pennington, Jeffrey, Richard Socher, and Christopher Manning (Oct. 2014). “**GloVe: Global Vectors for Word Representation**”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Alessandro Moschitti, Bo Pang, and Walter Daelemans. Doha, Qatar: Association for Computational Linguistics, pp. 1532–1543. DOI: 10.3115/v1/D14-1162. URL: <https://aclanthology.org/D14-1162>.